Access Control

Two roles on the Juniper Test Station: **operator** and **admin**.

The idea

An operator running a qualification sequence should never be blocked by a forgotten password. What needs protecting is the *definition* of a test, not the running of one.

So the station boots into **operator** and stays there. No login, no prompt, no shared sticky note on the monitor. Admin is a password-gated elevation of the same browser session, used when someone needs to change what the tests *are*.

	Operator	Admin
Login	none — the default	password
Connect the V71	✅ on saved settings	✅ and can change the settings
Run a saved test sequence	✅	✅
Abort / continue a run	✅	✅
View results, history, exports	✅	✅
Create / edit / retire sequences	❌	✅
Run arbitrary step parameters	❌	✅
Change connection settings	❌	✅
PVD verification	❌ (history visible)	✅
Thermal setpoints, vents, PID, PLC outputs	❌	✅
DC load configure / input on-off	❌	✅
PEC-0063 qualification runs	❌	✅

How an operator runs a test

The operator picks a **saved sequence** from a dropdown, fills in operator name, part number and DUT serial, and presses Run. The sequence's steps are shown read-only so they can see what is about to happen.

They never post step parameters. `POST /api/hipot/run_sequence` takes a `sequence_id`, and the step values come from the stored definition — there is no path from the operator's screen to an unreviewed test condition. The ad-hoc route, `POST /api/hipot/run`, accepts arbitrary voltages and dwell times and is admin-only. That split is the entire mechanism.

The session record notes which sequence and which revision produced it.

Sequences

An admin builds a sequence in the Sequence Builder, names it, and saves it. It appears in every operator's dropdown immediately.

- **Validation happens on save**, not on run, so a bad definition is caught by the admin who wrote it rather than by an operator at the bench with a DUT wired up.
- **Editing bumps a revision counter.** Changing a sequence after a batch has run should be visible, not silent.
- **Retiring is soft by default.** The sequence disappears from the operator's list but the definition stays, so a result recorded against it remains traceable. `DELETE ?hard=1` removes it outright.

The admin password

Stored as a **PBKDF2-SHA256 verifier** in `app_settings.admin_password_hash`:

```
pbkdf2_sha256$600000$<salt_hex>$<hash_hex>
```

600,000 iterations, per current OWASP guidance. Comparison uses `hmac.compare_digest`, so a wrong guess takes the same time to reject regardless of how many leading characters happened to be right.

The password itself is never written to the database, a config file, or a log. **The only copy lives in 1Password** — Juniper vault, item *"Juniper Test Station - Admin (hi-pot_UART)"*.

`hipot_results.db` is gitignored, so neither the verifier nor the session signing key reaches the repository.

First run

With no password configured, `admin_configured` is `false` and the Admin button reads *"Set up admin"*. Setting the first password is the one unauthenticated path — with no admin in existence there is nobody who could authorise creating one, and the station would otherwise be permanently operator-only. Every later change requires the current password.

The station does **not** fall open to admin when no password is set. It stays operator-only, which is the safe direction to fail.

Changing it

Log in as admin → Admin button → set a new password. Then update the 1Password item, or the two drift apart and nobody can get back in.

Idle timeout

An admin session drops back to operator after **30 minutes** without a request (`auth.ADMIN_IDLE_TIMEOUT_S`). The station sits unattended on a bench; an admin who walks away should not leave the settings unlocked behind them.

The timeout is enforced when the role is *read*, not by a background sweep, so a stale session can never be used even once after it expires. The UI re-checks every 60 seconds, so the button and the gated panels reflect reality rather than surfacing as a surprise 403 when someone finally clicks something.

How the UI hides things

The shared page chrome asks `/api/auth/status` and sets `data-role` on `<body>`. Anything gated carries `class="admin-only"`.

```
.admin-only{display:none !important;}
body[data-role="admin"] .admin-only{display:revert !important;}
```

Elements start hidden and are revealed — never the reverse. A failed or slow role check leaves the station locked down rather than briefly flashing the controls open.

This is presentation, not security. Every gated action is enforced server-side by the `@admin_required` decorator; hiding the controls just keeps the operator's screen honest about what they can do. Anyone can read the HTML; nobody can post to a gated route without the session.

API

Method	Endpoint	Description
GET	`/api/auth/status`	Current role, whether an admin password exists, idle timeout
POST	`/api/auth/login`	`{password}` → elevate this session to admin
POST	`/api/auth/logout`	Drop back to operator
POST	`/api/auth/set_password`	`{new_password, current_password?}`
GET	`/api/settings/connection`	Saved connection settings (any role)
POST	`/api/settings/connection`	Change them (admin)
GET	`/api/sequences`	Saved sequences (any role; `?all=1` includes retired, admin only)
GET	`/api/sequences/<id>`	One sequence
POST	`/api/sequences`	Create (admin)
PUT	`/api/sequences/<id>`	Update, bumps revision (admin)
DELETE	`/api/sequences/<id>`	Retire; `?hard=1` deletes (admin)
POST	`/api/hipot/run_sequence`	Run a saved sequence (any role)
POST	`/api/hipot/run`	Run ad-hoc steps (admin)

Gated routes return **403**, not 401. The operator is a legitimate, fully authenticated identity here — they are not unauthenticated, they are unauthorised.

Files

File	Role
`auth.py`	Password hashing, session roles, `@admin_required`
`database.py`	`app_settings` and `test_sequences` schema and CRUD
`app.py`	Auth routes, sequence CRUD, role-aware UI